

Creating Define-XML Solutions Using Python and odmlib

R/Pharma 2026 — Hands-On Workshop

Sam Hume

Agenda – 3 hours

#	Segment	Time
1	Introduction and environment setup	20 min
2	Block 1 – Read, explore, and report on a Define-XML v2.1	40 min
3	Block 2 – Generate a minimal Define-XML v2.1	40 min
4	Block 3 – Schema validation, ref/def checks, conformance	40 min
5	Demo – Extending the Define-XML model	10 min
6	Demo – The odmlib skill with generative AI	10 min
7	Demo – defineutils and other command-line tools	10 min
8	Q&A and conclusion	10 min

How the workshop works

Repo: github.com/swhume/r-pharma-odmlib

- One folder per segment: `01_read_explore/`, `02_create_define/`, ...
- `lecture_notes.md` — everything needed to solve the exercise
- **Exercise notebook** — working cells + `# YOUR CODE HERE` TODOs
- `solutions/` — completed, executed notebooks
- Self-paced, ~20 minutes, **stretch goals** for fast finishers

What is Define-XML?

The **machine-readable data definitions** for a clinical study — required by FDA and PMDA for SDTM, SEND, and ADaM data.

One define.xml describes:

- **Datasets, variables, and codelists**
- **Derivations** and **value-level metadata** with conditions
- **Documents** — aCRF, reviewer's guide, dataset file links

...all wired together as a **graph of cross-referenced metadata**.

What is odmlib?

Open-source (MIT) Python library: the **CDISC ODM family as schema-aware object models**. `pip install odmlib`

```
loader = LD.ODMLoader(DL.XMLDefineLoader(model_package="define_2_1"))
loader.open_odm_document("define.xml")
mdv = loader.root().Study.MetaDataVersion

age = mdv.find("ItemDef", "OID", "IT.DM.AGE")
print(age.Name, age.DataType, age.Origin[0].Type)
```

Elements are classes · attributes are typed · references are checkable

One library, the whole ODM family

Standard	Status
ODM 1.3.2	stable
ODM 2.0	draft
Define-XML 2.0 / 2.1	today's focus
Analysis Results Metadata (ARM) 1.0	stable
Dataset-XML 1.0.1 · CT-XML 1.1.1	stable
Dataset-JSON 1.1	stable

Learn the idioms once — they transfer 1:1 across standards.

Why not hand-roll the XML?

```
# hand-rolled: you own all of this
ET.register_namespace("def", "http://www.cdisc.org/ns/def/v2.1")
item = ET.SubElement(mdv, "ItemDef", {"OID": "IT.DM.AGE"}) # required attrs?
ref = ET.SubElement(igd, "ItemRef", {"ItemOID": "IT.DM.AGEE"}) # typo ships
```

```
# odmlib: the model owns it
item = DEF.ItemDef(OID="IT.DM.AGE", Name="AGE", DataType="integer")
igd.ItemRef.append(DEF.ItemRef(ItemOID="IT.DM.AGEE", Mandatory="No"))
odm.verify_oids(create_oid_checker("define_2_1"))
# OdmlibOIDError: OID IT.DM.AGEE referenced in ItemOID is not found
```

Namespaces · element order · required attributes · OID integrity —
hand-rolled failures are **silent**; odmlib failures are **loud**.

Where odmlib is used

An ecosystem of tools with odmlib as the engine:

- **CORE** - CDISC Open Rules Engine
- **CDISC 360i** - Define-XML and CRF generation from a USDM study design
- **defineutils** — Define-XML CLI: metrics, validate, refs, HTML
- **gendefine** — Define-XML ↔ metadata spreadsheets
- **odmlib_examples / odmlib_snippets** — runnable reference code
- **sponsor specific** - some sponsors have developed solutions using odmlib

...plus internal pipelines across sponsors, CROs, and vendors.

The odmlib development loop

```
load / build → modify → validate → serialize
                        |
                    1 XSD schema · 2 OID ref/def · 3 conformance · 4 element order
```

Validation-first: validate early, collect *every* error, fix, repeat.

The loop is the workshop's spine:

Block 1 = *load* · Block 2 = *build + serialize* · Block 3 = *validate + repair*

What's new in odmlib 0.2.1

- **Collect-all-errors validation** — `validate(collect_errors=True)` returns *every* finding across layers; `max_errors` caps runaway lists
- `to_element()` — real ElementTree interop
- **ARM 1.0 XSD bundled** + corrected ARM serialization order
- Define-XML `leaf` / `leafID` / `ArchiveLocationID` reference checks
- **The odmlib skill** — AI-assisted odmlib development (demo in segment 6)

AI-assisted odmlib development

The **odmlib skill**: curated instructions + API reference + runnable examples that load into an AI coding assistant (Claude Code).

- Triggers on the *domain* — even when odmlib isn't named
- Generates code with the **correct idioms** you'll learn today
- **Validation-first loop** — it checks its own output
- **Live demo in segment 6**

“ Generic AI writes *plausible* CDISC code —
the skill makes it **correct**, and validated.

”

What you'll build today

- **Block 1** — a metadata report mined from a realistic 179-variable MSG example define.xml
- **Block 2** — a schema-valid Define-XML v2.1, built from scratch in ~60 lines of Python
- **Block 3** — a broken define.xml diagnosed, repaired as objects, and re-validated to clean
- **Demos** — vendor extensions · AI-assisted development · CLI tools

Intermediate Python is plenty — all CDISC specifics are provided.

Environment setup — Posit Workbench

1. **Sign in** at workshop.posit.team —
Sign in with OpenID (new user? **Register** with first name + last initial)
or *Sign in with GitHub*
 2. **New Session** → **Positron** → default image, other defaults →
Start Session
 3. **New** → **New Folder from Git...** →
`https://github.com/swhume/r-pharma-odmlib`
 4. In the Positron **Terminal**: `pip install -r requirements.txt`
- AI:** choose the **Anthropic Claude Sonnet** model, and mind the usage limits.
Sessions do NOT persist after the workshop — save anything you want to keep.

Environment setup — JupyterLab

```
git clone https://github.com/swhume/r-pharma-odmlib.git
cd r-pharma-odmlib
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
jupyter lab
```

Full instructions + troubleshooting: [00_setup/setup_instructions.md](#)

Prefer VS Code or PyCharm notebooks? Both work — see the setup notes.

Verify your setup

Open `00_setup/verify_setup.ipynb` → **Run** → **Run All Cells**

Expected result:

```
Python: 3.12.x
odmlib: 0.2.1
Loaded study: TEST ODM ItemGroupDef
defineutils is installed
✅ You are ready for Block 1
```

Problems? See `setup_instructions.md` — or **drop it in the chat**.

Meet the data

`data/` ships realistic, submission-shaped metadata — not toy files:

- `defineV21-SDTM.xml` — the CDISC SDTM-MSG example:
11 datasets • 179 variables • 40 codelists • 33 methods
- **Deliberately broken variants** for Block 3 — each hides a *different class* of error (schema, references, conformance)
- `definev21-adam.xml` — ADaM + Analysis Results Metadata (ARM)

Provenance and details: `data/README.md`

Ground rules

- **Questions anytime** — drop them in the chat, I read it between beats
- Exercises are **self-paced** — stretch goals if you finish early
- The **solutions/** folder is not cheating — it's the safety net
- Everything is yours to keep: repo, data, notebooks, slides

Up next: Block 1 — Read, Explore, and Report